

Pseudocode und Textbeispiele

1. Diagnose-Pseudocode (deterministisch)

Eingang: `Sentence`, `orderedChunks`

Ausgang: genau ein `DiagnosisState`

1.1 Einstieg

None

```
function computeDiagnosis(sentence, chunks): Diagnosis
```

1.2 Prioritätsreihenfolge (hart)

None

```
if isPerformativeInstable(chunks):  
    return PERFORMATIV_INSTABIL  
  
if isNormativeSelfAnnulling(chunks):  
    return NORMATIV_SELBSTANNULLIEREND  
  
if isConflictual(chunks):  
    return KONFLIKTAER  
  
if isFormallyStableSemanticallyEmpty(chunks):  
    return FORMAL_STABIL_SEMANTISCH_LEER  
  
if isMulticore(chunks):  
    return MEHRKERNIG
```

```
return STABIL
```

1.3 Einzelprüfungen

● **performativ-instabil**

None

```
function isPerformativeInstable(chunks):  
    hasSelfReference = exists chunk where chunk.text contains  
        ["Satz", "Sprache", "Bedeutung", "Wahrheit",  
        "Ausdruck"]  
  
    hasNegation = exists chunk where chunk.text contains  
        ["unlesbar", "verpufft", "zerfällt", "Schweigen",  
        "ohne Bedeutung"]  
  
    return hasSelfReference AND hasNegation
```

● **normativ stabil / selbstannullierend**

None

```
function isNormativeSelfAnnulling(chunks):  
    hasNorm = count chunks of type "judgement.normative" >= 1  
    negatesNorm = exists chunk where chunk.text contains  
        ["nicht gültig", "ohne Grundlage", "existiert nicht",  
        "nichtig"]  
  
    return hasNorm AND negatesNorm
```

● **konfliktär**

None

```
function isConflictual(chunks):  
    norms = chunks of type "judgement.normative"
```

```

socials = chunks of type "evaluation.social"

if norms.length >= 2:
    return polarity(norms[0]) != polarity(norms[1])

if socials.length >= 2:
    return polarity(socials[0]) != polarity(socials[1])

return false

```

(Polarity kann im Prototyp hart gecodet sein: positiv / negativ via Keywordliste)

● formal stabil / semantisch leer

```

None
function isFormallyStableSemanticallyEmpty(chunks):
    subject = chunk of type "core.subject"

    if subject.text in ["es", "dies", "das"]:
        hasConcreteObject = exists chunk of type "core.object"
    OR "state"
        return NOT hasConcreteObject

    return false

```

● mehrkernig

```

None
function isMulticore(chunks):
    predicates = chunks of type "core.predicate"

    if predicates.length <= 1:
        return false

    subordinated = exists predicate with marker
        ["dass", "um zu", "weil", "während"]

```

```
return NOT subordinated
```

2. Konkrete JS / TS-Skizze (Prototype-fähig)

2.1 Typen

TypeScript

```
type DiagnosisState =  
  | "performativ_instabil"  
  | "normativ_selbstannullierend"  
  | "konfliktaer"  
  | "formal_stabil_semantisch_leer"  
  | "mehrkernig"  
  | "stabil";  
  
type ChunkType =  
  | "core.subject"  
  | "core.predicate"  
  | "core.object"  
  | "apposition"  
  | "relation"  
  | "state"  
  | "evaluation.social"  
  | "judgement.normative"  
  | "ornament";  
  
interface Chunk {  
  id: string;  
  type: ChunkType;  
  text: string;  
}
```

2.2 Diagnose-Entry

TypeScript

```
function computeDiagnosis(chunks: Chunk[]): DiagnosisState {
  if (isPerformativeInstable(chunks)) return
    "performativ_instabil";
  if (isNormativeSelfAnnulling(chunks)) return
    "normativ_selbstannullierend";
  if (isConflictual(chunks)) return "konfliktaer";
  if (isFormallyStableSemanticallyEmpty(chunks)) return
    "formal_stabil_semantisch_leer";
  if (isMulticore(chunks)) return "mehrkernig";
  return "stabil";
}
```

2.3 Prüfungen

TypeScript

```
function isPerformativeInstable(chunks: Chunk[]): boolean {
  const selfRefs = ["satz", "sprache", "bedeutung",
    "wahrheit"];
  const negations = ["unlesbar", "verpufft", "zerfällt",
    "schweigen"];

  const hasSelfRef = chunks.some(c =>
    selfRefs.some(k => c.text.toLowerCase().includes(k))
  );

  const hasNegation = chunks.some(c =>
    negations.some(k => c.text.toLowerCase().includes(k))
  );

  return hasSelfRef && hasNegation;
}
```

TypeScript

```
function isNormativeSelfAnnulling(chunks: Chunk[]): boolean {
  const norms = chunks.filter(c => c.type ===
    "judgement.normative");
}
```

```

    const negators = ["nicht gültig", "nichtig", "ohne
grundlage"];

    const negates = chunks.some(c =>
        negators.some(k => c.text.toLowerCase().includes(k))
    );

    return norms.length >= 1 && negates;
}

```

TypeScript

```

function isConflictual(chunks: Chunk[]): boolean {
    const norms = chunks.filter(c => c.type ===
"judgement.normative");
    const socials = chunks.filter(c => c.type ===
"evaluation.social");

    if (norms.length >= 2) {
        return polarity(norms[0]) !== polarity(norms[1]);
    }

    if (socials.length >= 2) {
        return polarity(socials[0]) !== polarity(socials[1]);
    }

    return false;
}

```

TypeScript

```

function isFormallyStableSemanticallyEmpty(chunks: Chunk[]):
boolean {
    const subject = chunks.find(c => c.type === "core.subject");
    if (!subject) return false;

    if (["es", "dies",
"das"].includes(subject.text.toLowerCase())) {

```

```

    const hasConcrete = chunks.some(c =>
      ["core.object", "state"].includes(c.type)
    );
    return !hasConcrete;
  }

  return false;
}

```

TypeScript

```

function isMulticore(chunks: Chunk[]): boolean {
  const predicates = chunks.filter(c => c.type ===
"core.predicate");
  if (predicates.length <= 1) return false;

  const subordinators = ["dass", "weil", "während", "um zu"];
  const subordinated = predicates.some(p =>
    subordinators.some(k => p.text.toLowerCase().includes(k))
  );

  return !subordinated;
}

```

TypeScript

```

function polarity(chunk: Chunk): "positive" | "negative" {
  const negative = ["nicht", "kein", "nie", "verweigert",
"scheitert"];
  return negative.some(k =>
chunk.text.toLowerCase().includes(k))
    ? "negative"
    : "positive";
}

```

Integration zwischen Re-Render und Diagnose

1. Integrationsprinzip (ein Satz)

Re-Render erzeugt eine neue formale Realität.
Diagnose bewertet ausschließlich diese Realität, niemals den Ursprungszustand.

Das heißt:

- keine Vorab-Diagnose
- keine Live-Diagnose
- keine Diagnose ohne Render

Diagnose ist **Folge**, nicht Begleiter.

2. Integrations-Ablauf (verbindlich)

2.1 Gesamtablauf (Sequenz)

None

User DnD

- Struktur ändert sich (order)
- User klickt „Neu linearisieren“
- Re-Render
- Invarianz-Check
- Diagnose
- Anzeige

Kein anderer Einstiegspunkt.

2.2 Harte Kopplung

- Diagnose **darf nicht** laufen, wenn:
 - Re-Render abgebrochen wurde
 - kein valider Prädikatskern existiert
 - Token-Invarianz verletzt ist

In diesen Fällen:

- **keine Diagnose**
 - Status: `nicht_renderbar`
-

3. Zustandsabhängige Wirkung auf Re-Render

Diagnose **verändert den gerenderten Satz nicht**.
Aber sie **begrenzt Folgeaktionen**.

3.1 Diagnose → UI-Verhalten (Mapping)

Diagnosezustand	Re-Render erlaubt	Weitere Analyse	Hinweise
stabil	✓	optional	max. 2
mehrkernig	✓	optional	max. 2
konfliktär	✓	✗	1
normativ_selbstannullierend	✓	✗	1
performativ_instabil	✓	✗	0

formal_stabil_semantisch_lee r	✓	optional	1
nicht_renderbar	✗	✗	1

Wichtig:

Kein Zustand blockiert Re-Render **rückwirkend**.

Blockade ist **diagnostisch**, nicht technisch.

4. Integrationslogik (Pseudocode)

None

```
function processSentence(sentence):

    if not sentence.orderChanged:
        return NO_OP

    renderResult = reRenderSentence(sentence)

    if renderResult.aborted:
        return {
            sentence: null,
            diagnosis: "nicht_renderbar"
        }

    diagnosis = computeDiagnosis(renderResult.chunks)
```

```
return {  
    sentence: renderResult.sentence,  
    diagnosis: diagnosis  
}
```

5. JS / TS – integrierte Pipeline

TypeScript

```
interface RenderPipelineResult {  
    sentence?: string;  
    diagnosis: DiagnosisState | "nicht_renderbar";  
}
```

TypeScript

```
function processSentence(  
    sentence: Sentence  
): RenderPipelineResult {  
  
    if (!sentenceHasStructuralChange(sentence)) {  
        return { diagnosis: "stabil" }; // oder leer anzeigen  
    }  
}
```

```
const render = reRenderSentence(sentence);

if (render.aborted) {
    return { diagnosis: "nicht_renderbar" };
}

const diagnosis = computeDiagnosis(render.chunks);

return {
    sentence: render.sentence,
    diagnosis
};
}
```

6. Integrierte Abbruchlogik (wichtig)

6.1 Re-Render-Abbruch

```
TypeScript

if (!hasValidPredicate(sentence)) {
    abort("no predicate");
}
```

6.2 Diagnose-Abbruch

TypeScript

```
if (render.aborted) {  
    skipDiagnosis();  
}
```

Keine Fallback-Diagnose.

Scheitern ist ein valider Zustand.

7. Diagnosen als „Sperrern“, nicht als Verbote

Diagnose beeinflusst **nur**:

- ob Drill-Down angeboten wird
- ob weitere Hinweise erscheinen
- ob der Zustand „final“ markiert wird

Beispiel:

TypeScript

```
function allowsDrillDown(state: DiagnosisState): boolean {  
    return ![  
        "performativ_instabil",  
        "konfliktaer",  
        "normativ_selbstannullierend"  
    ].includes(state);  
}
```

8. Wichtige Invarianzregel (systemisch)

**Der gleiche strukturelle Zustand
muss immer zur gleichen Diagnose führen.**

Deshalb:

- Diagnose **niemals** vom Ursprungs-Text
 - **nur** von:
 - Chunk-Typen
 - Chunk-Anordnung
 - Chunk-Inhalten
-

9. Minimaler Endzustand (MVP-relevant)

Ein MVP-Durchlauf endet immer in **einem** von drei sichtbaren Zuständen:

1. **Gerenderter Satz + Diagnose**
2. **Nicht renderbar**
3. **Keine strukturelle Änderung → kein Eingriff**

Mehr braucht es nicht.

10. Entscheidender Qualitätsindikator

Die Integration ist gelungen, wenn:

- derselbe Satz
- mit derselben Chunk-Anordnung
- **immer** dieselbe Diagnose erzeugt
- unabhängig davon, *wie* der Nutzer dorthin gekommen ist

Dann ist SPIN:

- reproduzierbar
 - erklärbar
 - vertrauenswürdig
-

15 Testsätze, bewusst unterschiedlich gebaut, jeweils mit klarem Zweck.

1. Stabil – einfach

Satz:

Der Hund schläft auf dem Teppich.

Testziel:

Baseline, einkernig, keine Zusatzfunktionen.

Erwartete Diagnose:

stabil

2. Stabil – mit Ornament

Satz:

Der alte Hund schläft ruhig auf dem weichen Teppich.

Testziel:

Ornamente vorhanden, aber nicht konfliktär.

Erwartete Diagnose:

stabil

3. Mehrkernig – gleichrangige Prädikate

Satz:

Der Hund schläft und beobachtet die Tür.

Testziel:

Zwei Prädikate ohne Unterordnung.

Erwartete Diagnose:

mehrkernig

4. Mehrkernig – verschachtelt, aber ungelöst

Satz:

Der Hund schläft, hört Geräusche und wartet.

Testziel:

Mehrere Kerne, keine klare Dominanz.

Erwartete Diagnose:

mehrkernig

5. Stabil – untergeordneter Nebensatz

Satz:

Der Hund schläft, während draußen der Regen fällt.

Testziel:

Relation klar untergeordnet.

Erwartete Diagnose:

stabil

6. Konfliktär – normative Gegensätze

Satz:

Man muss ehrlich sein, aber Lügen sind manchmal notwendig.

Testziel:

Zwei widersprüchliche normative Bewertungen.

Erwartete Diagnose:

konfliktaer

7. Konfliktär – soziale Bewertung

Satz:

Er gilt als zuverlässig und wird zugleich als untragbar beschrieben.

Testziel:

Soziale Zuschreibungen widersprechen sich.

Erwartete Diagnose:

konfliktaer

8. Formal stabil / semantisch leer

Satz:

Es ist so, wie es ist.

Testziel:

Platzhalter-Subjekt, keine externe Referenz.

Erwartete Diagnose:

formal_stabil_semantisch_leer

9. Formal stabil / semantisch leer (komplexer)

Satz:

Es geschieht, was geschieht, ohne dass etwas geschieht.

Testziel:

Zirkularität trotz formaler Korrektheit.

Erwartete Diagnose:

formal_stabil_semantisch_leer

10. Normativ stabil / selbstannullierend

Satz:

Diese Regel gilt, obwohl sie keinerlei Gültigkeit besitzt.

Testziel:

Norm wird gesetzt und zugleich negiert.

Erwartete Diagnose:

normativ_selbstannullierend

11. Normativ selbstannullierend (implizit)

Satz:

Man sollte Regeln befolgen, auch wenn es keine Regeln gibt.

Testziel:

Normativer Anspruch ohne Grundlage.

Erwartete Diagnose:

normativ_selbstannullierend

12. Performativ-instabil – explizit

Satz:

Dieser Satz hat keine Bedeutung.

Testziel:

Selbstreferenz + Bedeutungsentzug.

Erwartete Diagnose:

performativ_instabil

13. Performativ-instabil – poetisch

Satz:

Was ich hier sage, zerfällt im Moment des Sagens.

Testziel:

Performative Selbstauflösung ohne Metasprache.

Erwartete Diagnose:

performativ_instabil

14. Stabil – hoher Stil, aber konsistent

Satz:

Es war ein permanentes Zu-viel, das sich nicht über die Ohren, sondern direkt über das Gefühl ins Innere schob.

Testziel:

Hohe Dichte, aber klare Struktur.

Erwartete Diagnose:

stabil

15. Nicht renderbar (Strukturbruch)

Satz:

Der Hund, und schläft die Tür.

Testziel:

Kein rekonstruierbarer Prädikatskern.

Erwartete Diagnose:

nicht_renderbar

Wichtig für den Prototyp

Diese Liste deckt ab:

- **alle Diagnosezustände**
- **positive & negative Fälle**
- **Blockade vs. Handlungsfähigkeit**
- **Grenzfälle**, die dein Tool *aushalten muss*

Wenn **nur diese 15 Sätze** korrekt:

- gehunkt
- bewegt
- gerendert
- diagnostiziert

werden können,
ist dein MVP **technisch bestanden**.

Wenn du willst, kann ich als Nächstes:

- für **3–5 dieser Sätze** die **konkrete Chunk-Zerlegung** liefern
- oder **automatisierbare Unit-Test-Strukturen** skizzieren
- oder einen „**Worst-Case-Satz**“ bauen, der alles gleichzeitig triggert